

复旦大学计算机科学技术学院  
2017~2018 学年第 1 学期期末考试试卷

■ A 卷      □ B 卷

课程名称: Python 程序设计      课程代码: COMP110042.

开课院系: 计算机科学技术学院      考试形式: 闭卷

姓 名: \_\_\_\_\_ 学 号: \_\_\_\_\_ 专 业: \_\_\_\_\_

声明: 我已知悉学校对于考试纪律的严肃规定, 将秉持诚实守信宗旨, 严守考试纪律, 不作弊, 不剽窃; 若有违反学校考试纪律的行为, 自愿接受学校严肃处理。

学生(签名):

年      月      日

| 题号 | 一 | 二 | 三 | 四 1 | 四 2 | 总分 |
|----|---|---|---|-----|-----|----|
| 得分 |   |   |   |     |     |    |

一、单项选择题(20%) 请将答案填在题目前面的括号里。

- ( ) 关于 input 函数的使用, 错误的是哪一个?
  - 可以不传参数
  - 可以传一个参数
  - 若键入字母字符, 则返回一个字符串
  - 若键入数字字符, 则返回一个数值
- ( ) 下面关于//的用法, 错误的是哪一个?
  - 得到整除运算的商
  - 两个整数运算的结果是整数
  - 两个浮点数运算的结果是浮点数
  - 整数和浮点数运算的结果可能是整数也可能是浮点数, 看余数是否有小数部分
- ( ) 下列会发生短路运算的是哪个?
  - `1=='1' or 1==1.0 and True != False`
  - `(1=='1' or 1==1.0) and True != False`
  - `1==1.0 or 1=='1' and True != False`

D. (1=='1' or 1==1.0) and True != False

4. ( ) 假设 s 的内容是 'pie apple pie'，以下切片操作能得到 'pie' 的是：

A. s[:2]

B. s[1:3]

C. s[-3:-1]

D. s[-3:]

5. ( ) 假设下面的语句结构中 **statement1** 子句可能（也可能没有）引起一个异常，则以下描述错误的是：

try:

**statement1**

except **ExceptionA**:

**statement2**

except **ExceptionB**:

**statement3**

else:

**statement4**

finally:

**statement5**

**statement6**

A. 如果异常类型是 **ExceptionB**，**statement3** 会执行

B. 如果未发生异常，**statement4** 会执行

C. 如果该异常未被捕获，**statement5** 不会执行

D. 如果该异常未被捕获，**statement6** 不会执行

## 二、程序改错题（15%）请将答案填在题目后面的表格里。

在自然语言处理中，统计文本中单词出现的次数和找出文本中单词最经常使用的形式是两个常见的任务。在文本中，同样的单词会以不同的大小写形式出现，例如在一段文本中 `copy` 出现了 1 次，`Copy` 出现了 2 次，`COPY` 出现了 3 次，则单词 `copy` 出现的总次数为 6，同时单词 `copy` 最经常出现的形式为 `COPY`。

下面的程序 `stattoken.py` 实现了上述功能。其中，`loadText` 函数从 `LICENSE.txt` 文件中读入英语文本，通过正则式处理之后，分解成单词；`processTokens` 函数对已分解的单词，统计单词出现的总次数，并找出单词最经常使用的形式。

程序中有三处错误，请在后面的表格中填写错误行的标号（每行最右边注释里的数字。如果代码行后面没有标号或者有标号但后面附加\*，表示该行不是错误行）和改正后的整行代码（改正后的代码只能是一行，不允许跨越多行）。

```
from re import compile as recompile

def processTokens(words):                                     #1*
    #构造 dic 保存单词，例如:copy, Copy, COPY 各自出现的次数为 1,2,3,则
    #最终 dic 包含键为'copy'的键值对，即 dic['copy'] = {'copy':1, 'Copy':2, 'COPY':3}
    dic = dict()                                             #2
    for word in words:                                       #3
        word_lower = word.lower()                            #4
        case_dict = dic.get(word_lower, [])                 #5
        case_dict[word] = case_dict.get(word, 0) + 1        #6
        dic[word_lower] = case_dict                         #7
    #从已构造的 dic 中，统计出各单词出现的次数，结果在 fused_cases 中，
    #对应上述的例子，fused_cases['copy'] = 6;
    #同时，找出该单词最经常使用的形式，结果放在 standard_cases 中，
    #对应上述的例子，standard_cases['copy'] = 'COPY';
    fused_cases, standard_cases = {}, {}                     #8
    for word_lower, case_dict in dic.items():                 #9
        first = max(case_dict.items(), key=lambda x: x[1])[0] #10*
        fused_cases[word] = sum(case_dict.keys())            #11
        standard_cases[word_lower] = first                   #12
    return fused_cases, standard_cases                       #13*

def loadText():                                              #14*
    #从文件 LICENSE.txt 读取文本并合成一行
    with open('LICENSE.txt') as handler:                    #15
        text = ''.join(handler.readlines())                 #16
    #把文本中的',', '(', ')', ':', '"', '.', '/', 以及数字替换成 ' '
```

```

pattern = compile(r'[,()".\D:/]') #17
formatted_text = pattern.sub(' ', text) #18
#使用缺省参数的 split 函数分割文本
items = formatted_text.split() #19
return items #20*

if __name__ == '__main__': #21*
    words = loadText() #22
    wds, sts = processTokens(words) #23
    print("wordcounts: {} \n standard: {}".format(wds, sts)) #24

```

| 错误行位置 | 改正 |
|-------|----|
| 5     |    |
| 11    |    |
| 17    |    |

### 三、程序填空（15%）请将答案填在题目后面的表格里。

一个有效身份证号码验证规则如下：

1. 将身份证号码前 17 位数分别乘以不同的系数。从第一位到第十七位的系数分别为：7 9 10 5 8 4 2 1 6 3 7 9 10 5 8 4 2
2. 将这 17 位数字和系数相乘的结果相加。计算出来的和除以 11，看余数是多少？
3. 余数只可能有 0 1 2 3 4 5 6 7 8 9 10 这 11 个数字，其分别对应的最后一位身份证的号码为 1 0 X 9 8 7 6 5 4 3 2。即如果余数是 2，就会在身份证的第 18 位数字上出现罗马数字的 X。如果余数是 10，身份证的最后一位号码就是 2。

下面程序 IDverify.py 程序将根据以上规则验证用户输入的身份证号是不是一个有效的身份证号。

```
import string
factor=(7,9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2)
last=("1", "0", "X", "9", "8", "7", "6", "5", "4", "3", "2")
```

```
def charsAreNum(ID):
    '''验证前 17 位均为数字'''
    for char in _____1_____:
        if char not in string.digits:
            return False
    return True
```

```
while True:
    ID=input('请输入身份证号，0 则退出:')
    if ID=='0':
        break
    elif len(ID)!=18:
        print('输入位数不对，请重新输入！')
        continue
    elif not charsAreNum(ID):
```

```

print('输入的身份证前 17 位中包含不是数字的字符，请重新输入！')
continue
else:
    sum=0
    for i in range(17):
        _____2_____
    m=sum%11
    lastchar=_____3_____
    lastchar=lastchar.upper()
    if lastchar==last[m]:
        print(ID,'为合法身份证号码!')
    else:
        print(ID,'为非法号码')

```

程序中有三处代码不完整，请在后面的表格中填写完整。

| 序号 | 填空 |
|----|----|
| 1  |    |
| 2  |    |
| 3  |    |

## 四、编程（50%）

### 1、（25%）

以  $e$  为底的幂函数  $e^x$  可以按照如下级数展开公式计算：

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

请编写程序，完成如下功能：

- 1) 提示用户输入一个数值（可以是整数或者浮点数）。如果用户输入正确，则按照上述公式计算并输出，然后继续提示用户输入一个数值。这个过程一直重复，直到用户仅仅输入了回车时为止，此时程序正常退出。如果用户输入的不是一个数值，则提示用户输入有误。

- 2)  $e^x$  不允许使用 `math` 模块中的相关函数来计算，而是使用上述公式进行求和。

显然在求和时不能无限制地计算，而是要求新加进去的项  $\frac{x^n}{n!}$  小于等于  $10^{-10}$  为止。

下面给出了参考的运行示例：

```
请输入一个数值...1
e**1 = 2.71828182845823
请输入一个数值...2
e**2 = 7.389056098925863
请输入一个数值...1/2
您的输入有误: could not convert string to float: '1/2'
请输入一个数值...0.5
e**0.5 = 1.648721270699598
请输入一个数值...-1.5
e**-1.5 = 0.2231301601509859
请输入一个数值...
bye....
```

考察： 如何使用循环结构以及异常处理等来接受用户输入；如何使用循环结构对于级数求和问题进行求解。

2、（25%）文件 ‘scores.csv’ 保存了某次考试批阅试卷后产生的考试分数。下面给出了该文件的前面几行内容：

```
0858, St0858, 19, 19. 2, 14, 11, 7. 5, 12, 8. 4, 13
0875, St0875, 12, 16, 7. 9, 14, 5, 12. 4, 5, 12
0198, St0198, 6, 12. 5, 11, 8. 3, 10, 11, 12, 6. 3
0946, St0946, 18, 13. 2, 13. 3, 10. 6, 5. 6, 15. 0, 12. 1, 10
0992, St0992, 18. 8, 8, 9. 9, 12, 12. 1, 7, 5, 10. 4
```

该文件中的每行对应了某个学生，包括 10 个字段，这些字段之间以逗号(,)隔开。第一个字段为学生的学号(ID)，第二个字段为学生的姓名(NAME)，接下来 8 个字段为这次考试的 8 道大题的得分，其中后面 4 道大题为四选二，即其中 2 道分数最多的题目计入到最终分数中。比如学号为 0858 的同学，后面 4 道题中选择分组最高的两题(12,13)，总分为 88.2。

请计算出每个学生的总分，并且根据总分给出相应的等级，最后输出相应的结果，包括学生学号、姓名、总分和等级，总分保留小数点后两位，注意最后的输出中各列要能够对齐。为了简单起见，假设姓名中不包含中文字符。成绩等级根据总分所在的区间判断：

- ✓ 如果总分在[85,100]，则等级为 A
- ✓ 如果总分在[75,85)，则等级为 B
- ✓ 如果总分在[65,75)，则等级为 C
- ✓ 如果总分在[60,65)，则等级为 D
- ✓ 如果总分在[0,60)，则等级为 F
- ✓ 否则等级为?

下面给出了参考的运行示例，仅仅列出了前面若干行的内容：

| ID   | Name   | Score | Rank |
|------|--------|-------|------|
| 0858 | St0858 | 88.20 | A    |
| 0875 | St0875 | 74.30 | C    |
| 0198 | St0198 | 60.80 | D    |
| 0946 | St0946 | 82.20 | B    |

考察点：文件的读，如何分割字符串，如何利用列表推导式或循环将字符串转变为浮点数，求和与 sorted，多分支结构，如何格式化输出。  
如果觉得内容多，可以去掉成绩等级的判断部分。